UNIVERSIDAD DISTRITAL FRANCISCO JOSÉ DE CALDAS

FACULTAD DE INGENIERÍA
Ingeniería de Sistemas

Project Definition Document

Data and analisys programing

Workshop #1

Students

Juan David Amaya Patiño 20221020057
Juan Pablo Mosquera 20221020026
Edward Jualian Garcia Gaitan 20212020136

Professor: Carlos Andrés Sierra Virguez

Date: May 2026

Contents

1 Project Overview **3**
1.1 Motivation . . . 3
1.2 Goal . . . 3
1.3 Scope and Limitations . . . 3

2 User Definition **4**
2.1 Profile . . . 4
2.2 User Stories . . . 5

3 Source Research **6**
3.1 API: The Movie Database (TMDb) . . . 6
3.2 Web Scraping: Rotten Tomatoes . . . 9

4 Data Samples **12**

5 Extraction Strategy **13**

6 Pipeline Architecture Overview **14**
6.1 Architecture Overview . . . 14
6.2 Layered Data Model . . . 15
6.3 Orchestration Strategy . . . 16
6.4 Technologies and Components . . . 16
6.5 End-to-End Analytical Flow . . . 16

1 Project Overview

1.1 Motivation

Users constantly share their opinions about movies, series and content releases, generating a large volume of textual data that reflects public perception in near real time.

Understanding this public sentiment is relevant also for analysts and researchers interested in social behavior and trends. Platforms such as IMDb and Rotten Tomatoes provide a rich source of user reviews and ratings, which makes them ideal for studying how audiences react to different types of content.

This data is often unstructured, noisy and sometimes inconsistent, which makes its analysis challenging. This project is motivated by the opportunity to apply data engineering techniques, web scraping and API consumption, together with NLP methods, to extract insights from this kind of data.

1.2 Goal

The main goal of this project is to design and implement a data pipeline capable of collecting, processing and analyzing public sentiment related to streaming and entertainment content.

the project aims to:

- Extract user reviews and ratings from IMDb and Rotten Tomatoes using web scraping techniques.
- Complement the dataset with additional structured data obtained from APIs when available.
- Clean and transform the collected data into a usable format for analysis.
- Apply Natural Language Processing NLP techniques to classify
- Generate aggregated insights that can be visualized in a simple dashboard.

By the end of the project, we expect to have a functional prototype that shows public sentiment analyzed through an end to end data pipeline. While it may not be a production level system, it should be robust enough to show the potential of these techniques in a real world scenario.

1.3 Scope and Limitations

This project is developed within the context of an academic course, so its scope is intentionally limited.

In terms of scope, the project will:

- Focus only on selected content rather than the entire catalog of streaming platforms.
- Use IMDb and Rotten Tomatoes as the primary data sources.
- Implement a basic but functional pipeline (Bronze, Silver, Gold layers).

- Perform sentiment analysis using standard NLP techniques.

On the other hand, some limitations must be considered:

- Web scraping may be restricted by site structure changes or access limitations.
- The dataset size will be relatively small compared to real industry systems.
- Sentiment classification may not be perfectly accurate, especially with sarcasm or ambiguous reviews.

the project aims to provide a solid approximation analysis, offering a practical learning experience in data engineering and NLP workflows.

2 User Definition

2.1 Profile

The primary user of this system is a Brand Analyst or Content Analyst working in the entertainment or streaming industry. This type of user is responsible for monitoring public perception of movies, series and streaming platforms, in order to support strategic and marketing decisions.

Technical Level: The user is expected to have an intermediate technical level. They are familiar with data visualization tools and basic data interpretation, but they are not necessarily experts in programming or data engineering. Therefore, the system should present insights in a clear and accessible way, without requiring deep technical knowledge to understand the results.

Information Needs: The user needs to quickly understand how audiences perceive specific content. This includes:

- Overall sentiment
- Trends in audience opinion over time
- Distribution of ratings and reviews
- Identification of highly positive or highly negative reactions

Additionally, the user may be interested in comparing different titles or detecting sudden changes in public opinion, for example after a release or a controversial event.

Decision-Making Context: The insights generated by this system can support decisions such as:

- Evaluating the success or reception of a movie or series
- Adjusting marketing or communication strategies
- Identifying potential risks related to negative public perception

- Understanding audience preferences and trends

These decisions are typically made in a fast paced environment where having timely and summarized information is important even if the data is not perfect

Justification: This user benefits from the sentiment analysis pipeline and dashboard because it transforms large volumes of unstructured textual data into actionable insights Instead of manually reading hundreds of reviews, the analyst can rely on aggregated metrics and visualizations to quickly assess public opinion.

Moreover, the pipeline automates data collection and processing, reducing manual effort and allowing the user to focus on interpretation rather than data gathering. While the system may not capture every nuance of human language, it provides a practical and scalable way to understand general sentiment trends

2.2 User Stories

User Story 1: View Overall Sentiment

Priority: High

User Story:
As a *brand analyst*, I want to see the overall sentiment of a movie or series, so that I can quickly understand public perception.

Acceptance Criteria:
Given that the user selects a specific title,
When the system processes available reviews,
Then the dashboard displays the overall sentiment (positive, neutral, negative) clearly.

User Story 2: Compare User vs Critic Ratings

Priority: High

User Story:
As a *Brand analyst*, I want to compare user ratings with official critic ratings across different sources, so that I can identify differences in perception between audiences and professional reviewers.

Acceptance Criteria:
Given that the system contains ratings from users and critics,
When the user selects a movie or series,
Then the system displays a comparison between user ratings and critic scores from both data sources.

User Story 3: Identify Extreme Opinions

Priority: Medium

User Story:
As a *brand analyst*, I want to identify highly positive and highly negative reviews, so that I can understand strong audience reactions.

Acceptance Criteria:
Given that reviews are classified by sentiment,
When the user filters by sentiment intensity,
Then the system shows the most extreme positive and negative comments.

User Story 4: Compare Multiple Titles

Priority: High

User Story:
As a *content analyst*, I want to compare sentiment across different movies or series, so that I can evaluate their relative performance.

Acceptance Criteria:
Given that multiple titles are selected,
When the user requests a comparison,
Then the system displays comparative metrics and visualizations for each title.

User Story 5: View Top Content by Release Year

Priority: Medium

User Story:
As a *brand analyst*, I want to view the best rated movies or series based on their release year, so that I can identify trends in high performing content over time.

Acceptance Criteria:
Given that the system contains ratings and release dates for different titles,
When the user selects a specific year,
Then the system displays the top-rated content released in that year.

---

3 Source Research

3.1 API: The Movie Database (TMDb)

Name: The Movie Database (TMDb) API
Base URL: `https://api.themoviedb.org/3`

Authentication

The TMDb API uses an **API key-based** authentication system. Each request must include a valid API key as a query parameter:

```
https://api.themoviedb.org/3/movie/popular?api_key=YOUR_API_KEY
```

This key is obtained by creating an account and registering an application on the TMDb developer platform.

Available Endpoints

Method	Endpoint	Description
GET	/movie/popular	Retrieve a list of currently popular movies
GET	/movie/top_rated	Retrieve top-rated movies of all time
GET	/movie/{id}	Detailed metadata for a specific movie
GET	/tv/popular	Retrieve a list of currently popular TV shows
GET	/tv/top_rated	Retrieve top-rated TV shows of all time
GET	/tv/{id}	Detailed metadata for a specific series
GET	/search/movie?query=	Search movies by title keyword
GET	/search/tv?query=	Search TV shows by title keyword

Table 1: Endpoints available in the **TMDb API v3**

Rate Limits

The TMDb API does not enforce strict rate limits at this time; however, an approximate upper threshold of ∼**40 requests per second** applies to prevent high-volume bulk scraping. According to their documentation, this limit is subject to change without prior notice and should be taken into account when designing the data extraction strategy.

Response Format

The API returns data in **JSON** format with structured fields including movie title, release date, average rating, vote count, and genre tags. The example below shows a representative response from the /tv/{id} endpoint:

```
{'page': 1,
 'results': [{'adult': False,
              'backdrop_path': '/tsRy63Mu5cu8etL1X7ZLyf7UP1M.jpg',
              'first_air_date': '2008-01-20',
              'genre_ids': [18, 80],
              'id': 1396,
              'name': 'Breaking Bad',
              'origin_country': ['US'],
              'original_language': 'en',
              'original_name': 'Breaking Bad',
              'overview': 'Walter White, a New Mexico chemistry teacher, is '
                          'diagnosed with Stage III cancer and given a '
                          'prognosis of only two years left to live. He '
                          'becomes filled with a sense of fearlessness and an '
                          "unrelenting desire to secure his family's financial "
                          'future at any cost as he enters the dangerous world '
                          'of drugs and crime.',
              'popularity': 153.7273,
              'poster_path': '/ztkUQFLlC19CCMYHW9o1zWhJRNq.jpg',
              'vote_average': 8.938,
              'vote_count': 17346},
             {'adult': False,
              'backdrop_path': '/rBOnrVlck7BIlGeWVlzYiZeg4l2.jpg',
              'first_air_date': '2023-09-29',
              'genre_ids': [16, 10759, 18, 10765],
              'id': 209867,
              'name': "Frieren: Beyond Journey's End",
              'origin_country': ['JP'],
              'original_language': 'ja',
              'original_name': '葬送のフリーレン',
              'overview': 'Decades after her party defeated the Demon King, an '
                          "old friend's funeral launches the elf wizard "
                          'Frieren on a journey of self-discovery.',
              'popularity': 94.2639,
              'poster_path': '/dqZENchTd7lp5zht7BdlqM7RBhD.jpg',
              'vote_average': 8.8,
              'vote_count': 728},
```


Figure 1: Sample JSON response from the TMDb API (`/tv/{id}` endpoint)

JSON format from `https://api.themoviedb.org/3/movie//movie/top_rated` endpoint :

Field	Type	Description
`id`	Integer	Unique movie identifier
`title`	String	Movie title (localized)
`original_title`	String	Title in original release language
`release_date`	String	Release date in `YYYY-MM-DD` format
`vote_average`	Float	Average user rating (0–10 scale)
`vote_count`	Integer	Total number of votes submitted
`popularity`	Float	TMDb internal popularity score
`genre_ids`	Array	List of genre identifiers
`overview`	String	Short plot synopsis

Table 2: Key fields extracted from the TMDb API JSON response

Justification of Use

The TMDb API serves as the **primary structured data source** for this project. Compared to web scraping, it offers several advantages:

- **Reliability:** Data is consistently formatted and maintained by an official provider.
- **Ease of integration:** JSON responses map directly to pipeline data models, reducing transformation overhead.
- **Rich metadata:** Fields such as genre, release date, language, and vote statistics complement the sentiment data extracted from reviews.
- **Complementarity:** It provides a structured backbone that anchors the unstructured review data collected from IMDb and Rotten Tomatoes through scraping.

3.2 Web Scraping: Rotten Tomatoes

Name: Rotten Tomatoes — Movies in Theaters
Target URL: `https://www.rottentomatoes.com/browse/movies_in_theaters/sort:popular`

robots.txt Verification

Prior to extraction, the `robots.txt` file at `https://www.rottentomatoes.com/robots.txt` was reviewed to confirm compliance with the site's crawling directives:

```
User-agent:  *
Disallow:  /m/*/pictures
Disallow:  /tv/*/pictures
Disallow:  /search
Disallow:  /critics/self-submission/
```

The `/browse/` path used in this project is not listed under any `Disallow` directive, confirming that access is permitted for general user agents.

HTML Structure

The page renders movie data using custom Web Components and `data-qa` attributes. Each movie is enclosed in a repeating card block identifiable by the following selectors:

Element	Selector / Attribute	Role
`div`	`data-qa="discovery-media-list-item"`	Root container for each movie card
`tile-dynamic`	`skeleton, isvideo`	Movie tile wrapper (poster + scores)
`rt-img`	`class="posterImage"`	Poster image CDN URL
`rt-text`	`slot="criticsScore"`	Tomatometer critics score
`rt-text`	`slot="audienceScore"`	Audience Popcornmeter score
`span`	`data-qa="discovery-media-list-item-title"`	Movie title text
`span`	`data-qa="discovery-media-list-item-start-date"`	Release / opening date
`a`	`data-qa="discovery-media-list-item-caption"`	Link to movie detail page

Table 3: HTML selectors used to extract data from Rotten Tomatoes

Extracted Fields

Field	Type	Description
`title`	String	Display title of the movie
`release_date`	String	Opening or re-release date
`critics_score`	String (%)	Tomatometer rating (may be absent)
`audience_score`	String (%)	Popcornmeter rating (may be absent)
`poster_url`	URL	CDN link to the movie poster image
`url`	URL	Full link to the movie detail page

Table 4: Fields extracted via BeautifulSoup from Rotten Tomatoes

A total of **68 records** were successfully extracted and exported to `rotten_tomatoes_movies.csv`.

Justification of Use

Rotten Tomatoes serves as the **primary review aggregation source** for this project. Web scraping was chosen over an API-based approach due to the following reasons:

- **No official public API:** Rotten Tomatoes does not currently offer a freely accessible API for bulk movie data retrieval.
- **Rich aggregated scores:** The platform provides both critic (Tomatometer) scores, enabling comparative sentiment analysis.
- **Compliance:** The scraped path (`/browse/`) is explicitly permitted by the site's `robots.txt` directives, and no authentication or paywalls were bypassed.
- **Complementarity:** The structured scores extracted here complement the qualitative metadata provided by the TMDb API, enriching the final dataset with audience perception indicators.

4 Data Samples

API Sample — TMDb `/movie/top_rated`

id	title	release_date	vote_avg	vote_count	popularity	lang
278	The Shawshank Redemption	1994-09-23	8.717	29953	69.70	en
238	The Godfather	1972-03-14	8.687	22626	42.33	en
240	The Godfather Part II	1974-12-20	8.572	13699	21.50	en
424	Schindler's List	1993-12-15	8.567	17223	23.70	en
389	12 Angry Men	1957-04-10	8.556	9823	19.44	en
1181678	¿Quieres ser mi hijo?	2023-09-21	8.536	303	8.24	en
129	Spirited Away	2001-07-20	8.500	18077	32.85	ja
155	The Dark Knight	2008-07-16	8.527	35349	32.37	en
19404	Dilwale Dulhania Le Jayenge	1995-10-20	8.517	4566	11.08	hi
497	The Green Mile	1999-12-10	8.505	18961	25.14	en
122	The Lord of the Rings: The Return of the King	2003-12-17	8.494	26189	30.40	en
496243	Parasite	2019-05-30	8.493	20290	28.61	ko
680	Pulp Fiction	1994-09-10	8.486	29849	29.06	en
372058	Your Name.	2016-08-26	8.479	12358	25.03	ja
157336	Interstellar	2014-11-05	8.500	39154	52.44	en
429	The Good, the Bad and the Ugly	1966-12-23	8.467	9489	16.91	it
13	Forrest Gump	1994-06-23	8.463	29388	26.03	en
769	GoodFellas	1990-09-12	8.452	14119	27.88	en
346	Seven Samurai	1954-04-26	8.451	4141	13.82	ja
12477	Grave of the Fireflies	1988-04-16	8.400	6409	0.01	ja

Table 5: First 20 records from the TMDb API — `/movie/top_rated` endpoint

Full dataset exported to `top_rated_movies.csv` *— fields include:* `id`, `title`, `original_title`, `release_date`, `vote_average`, `vote_count`, `popularity`, `genre_ids`, `overview`.

Scraping Sample — Rotten Tomatoes `/browse/movies_in_theaters`

title	release_date	critics_score	audience_score
Project Hail Mary	Mar 20, 2026	95%	97%
One Battle After Another	Sep 26, 2025	94%	85%
Ready or Not 2: Here I Come	Mar 20, 2026	76%	89%
Hoppers	Mar 06, 2026	93%	94%
undertone	Mar 13, 2026	73%	51%
The Secret Agent	Nov 26, 2025	98%	83%
Reminders of Him	Mar 13, 2026	55%	88%
Hamnet	Dec 05, 2025	87%	93%
THE BRIDE!	Mar 06, 2026	57%	70%
Scream 7	Feb 27, 2026	31%	76%
Wuthering Heights	Feb 13, 2026	57%	76%
Sentimental Value	Nov 07, 2025	95%	94%
Crime 101	Feb 13, 2026	88%	85%
Zootopia 2	Nov 26, 2025	91%	95%
GOAT	Feb 13, 2026	84%	93%
Slanted	Mar 13, 2026	68%	68%
Tow	Mar 20, 2026	80%	N/A
Send Help	Jan 30, 2026	93%	87%
Blue Moon	Oct 24, 2025	90%	76%
Mr. Nobody Against Putin	Jan 21, 2026	100%	76%

Table 6: First 20 records scraped from Rotten Tomatoes — Movies in Theaters (sorted by popularity)

Full dataset exported to `rotten_tomatoes_movies.csv` *— 68 records total, fields include:* `title`, `release_date`, `critics_score`, `audience_score`, `poster_url`, `url`.

5 Extraction Strategy

The extraction strategy of this project is based on a hybrid approach that combines API-based ingestion and controlled web scraping. This decision follows directly from the source research previously conducted: TMDb was selected as the primary structured source due to its stable JSON interface and official API access, while Rotten Tomatoes was selected as a

complementary perception-oriented source because it provides critic and audience scores that are publicly visible through the website.

For TMDb, extraction will be performed through authenticated HTTP GET requests using an API key. The pipeline will query selected endpoints such as `/movie/popular`, `/movie/top_rated`, and specific title-detail endpoints when necessary. Since TMDb returns structured JSON responses, the extraction process is straightforward: send the request, validate the response code, parse the returned JSON, and persist the full raw payload in the Bronze layer. The most relevant fields for the project include `id`, `title`, `original_title`, `release_date`, `vote_average`, `vote_count`, `popularity`, `genre_ids`, and `overview`. This source constitutes the structured backbone of the dataset.

For Rotten Tomatoes, extraction will be performed through web scraping because no freely accessible public API is available for this use case. Based on the source research, the selected path corresponds to the public `/browse/` section, which is not disallowed by the site's `robots.txt` file. The scraper will send HTTP requests to the selected page, retrieve the HTML, and parse the repeated movie-card structure using selectors already identified during source inspection. The extracted fields include `title`, `release_date`, `critics_score`, `audience_score`, `poster_url`, and `url`. The resulting records will then be exported in raw format for later processing.

From an operational perspective, the extraction process is batch-oriented. Rather than collecting data in real time, the system will run periodic extraction jobs during each analysis cycle. This is sufficient for the academic scope of the project and reduces implementation complexity. In practice, each execution of the extraction workflow will generate raw files from both sources and store them using a timestamp-based naming convention such as `source_topic_YYYYMMDD_HHMMSS.json` or `source_topic_YYYYMMDD_HHMMSS.csv`. This convention improves traceability, reproducibility, and downstream automation.

The strategy also includes basic reliability safeguards. API responses must be validated before parsing, missing fields should be tolerated gracefully, and extraction logs should record timestamps and source identifiers. In the case of TMDb, request frequency should remain within reasonable usage limits. In the case of Rotten Tomatoes, HTML selectors must be documented because structural changes in the site may affect scraper stability over time.

Overall, the extraction strategy transforms the findings of the source research into an actionable data acquisition plan. TMDb provides structured metadata through official API access, while Rotten Tomatoes contributes complementary public perception indicators through controlled scraping. Together, both extraction methods provide the raw inputs required by the rest of the pipeline. The raw outputs generated by this extraction strategy are later integrated into the pipeline architecture, where they are stored, transformed, and aggregated through the Bronze, Silver, and Gold layers.

6 Pipeline Architecture Overview

6.1 Architecture Overview

The proposed system follows a layered data pipeline architecture designed to transform raw entertainment data into analytical outputs that can be consumed by the final user. In consis-

tency with the workshop requirements, the architecture is organized into three main layers: Bronze, Silver, and Gold. This structure separates raw ingestion, data standardization, and business-oriented analytics, making the pipeline easier to understand, maintain, and extend.

The architecture integrates the two sources defined in the previous sections. TMDb provides structured metadata through an official API, while Rotten Tomatoes contributes public perception indicators obtained through controlled scraping. Once extracted, both sources enter the same pipeline, where they are stored in raw format, transformed into consistent analytical datasets, and later aggregated into dashboard-ready outputs.

6.2 Layered Data Model

Bronze Layer

The Bronze layer stores raw data exactly as it is obtained from the extraction process. For TMDb, this includes JSON responses returned by the API. For Rotten Tomatoes, this includes the raw structured output generated by the scraper, stored as JSON or CSV depending on the extraction process. At this level, no major transformations are applied beyond basic persistence and file organization.

The purpose of the Bronze layer is to preserve source integrity, ensure traceability, and support reprocessing if errors appear in later stages. Each ingestion run generates timestamped raw files using a naming convention such as `source_topic_YYYYMMDD_HHMMSS.json` or `source_topic_YYYYMMDD_HHMMSS.csv`.

Silver Layer

The Silver layer contains cleaned, normalized, and validated data. In this stage, records from TMDb and Rotten Tomatoes are transformed into a more consistent schema suitable for analysis. Typical operations include column renaming, date normalization, missing-value handling, duplicate removal, type conversion, and standardization of score fields. For example, percentage strings from Rotten Tomatoes can be converted into numeric values, while dates from both sources can be unified into a common format.

This layer also prepares selected textual attributes for later NLP-related analysis. Although the full sentiment processing may occur in a later stage, the Silver layer ensures that the underlying data is already clean and analytically usable. Processed outputs in this layer are intended to be stored in Parquet format for improved efficiency and compatibility with downstream tasks.

Gold Layer

The Gold layer contains aggregated and analysis-ready datasets designed for direct use in the dashboard. Instead of storing raw or record-level technical data, this layer focuses on metrics and summaries that respond to user needs defined in the project. These outputs may include title-level sentiment summaries, critic-versus-audience comparisons, temporal trends, and ranked lists of highly positive or highly negative content.

In this way, the Gold layer acts as the interface between data engineering processes and decision-support visualization. It transforms processed data into compact, interpretable outputs that can be directly consumed by the functional user.

6.3 Orchestration Strategy

The orchestration of the pipeline is based on Apache Airflow. A first DAG manages the ingestion stage and is responsible for triggering the extraction workflows defined in the previous section. This includes sending API requests to TMDb, launching the Rotten Tomatoes scraping task, and storing the resulting raw files in the Bronze layer.

Once new raw files are detected, a second workflow can be activated through a `FileSensor`. This second DAG handles the Silver-layer processing tasks, including cleaning, normalization, and transformation into Parquet datasets. After the Silver outputs are available, additional tasks generate the aggregated Gold datasets that will later feed the dashboard.

This orchestration strategy separates ingestion from transformation and aggregation, which improves modularity and simplifies maintenance. It also supports a more robust workflow, since failures in one stage can be diagnosed without affecting the entire system design.

6.4 Technologies and Components

The pipeline combines several technologies according to the role of each stage. Python is used as the main implementation language for extraction and transformation. TMDb data is acquired through HTTP requests, while Rotten Tomatoes data is obtained using scraping libraries such as `requests` and `BeautifulSoup`. For processing, Pandas or Polars can be used to clean and reshape the data in the Silver layer. If needed, PySpark remains a possible alternative for larger-scale transformations.

Apache Airflow is used to orchestrate the workflow, Docker is used to containerize the environment and improve reproducibility, and Plotly Dash is proposed as the visualization layer for the final analytical interface. Together, these components form a complete end-to-end architecture aligned with the academic objectives of the workshop.

6.5 End-to-End Analytical Flow

From an end-to-end perspective, the pipeline begins with the extraction of metadata and review-related indicators from external sources. Those raw outputs are first preserved in the Bronze layer, then cleaned and standardized in the Silver layer, and finally aggregated in the Gold layer into user-oriented indicators. The dashboard consumes these Gold datasets to present clear visual summaries for a Brand Analyst or Content Analyst interested in understanding public opinion about movies or series.

This architecture ensures that the project is not limited to isolated scripts or disconnected datasets. Instead, it establishes a coherent analytical workflow in which source acquisition, data engineering, and visualization are integrated into a single pipeline.

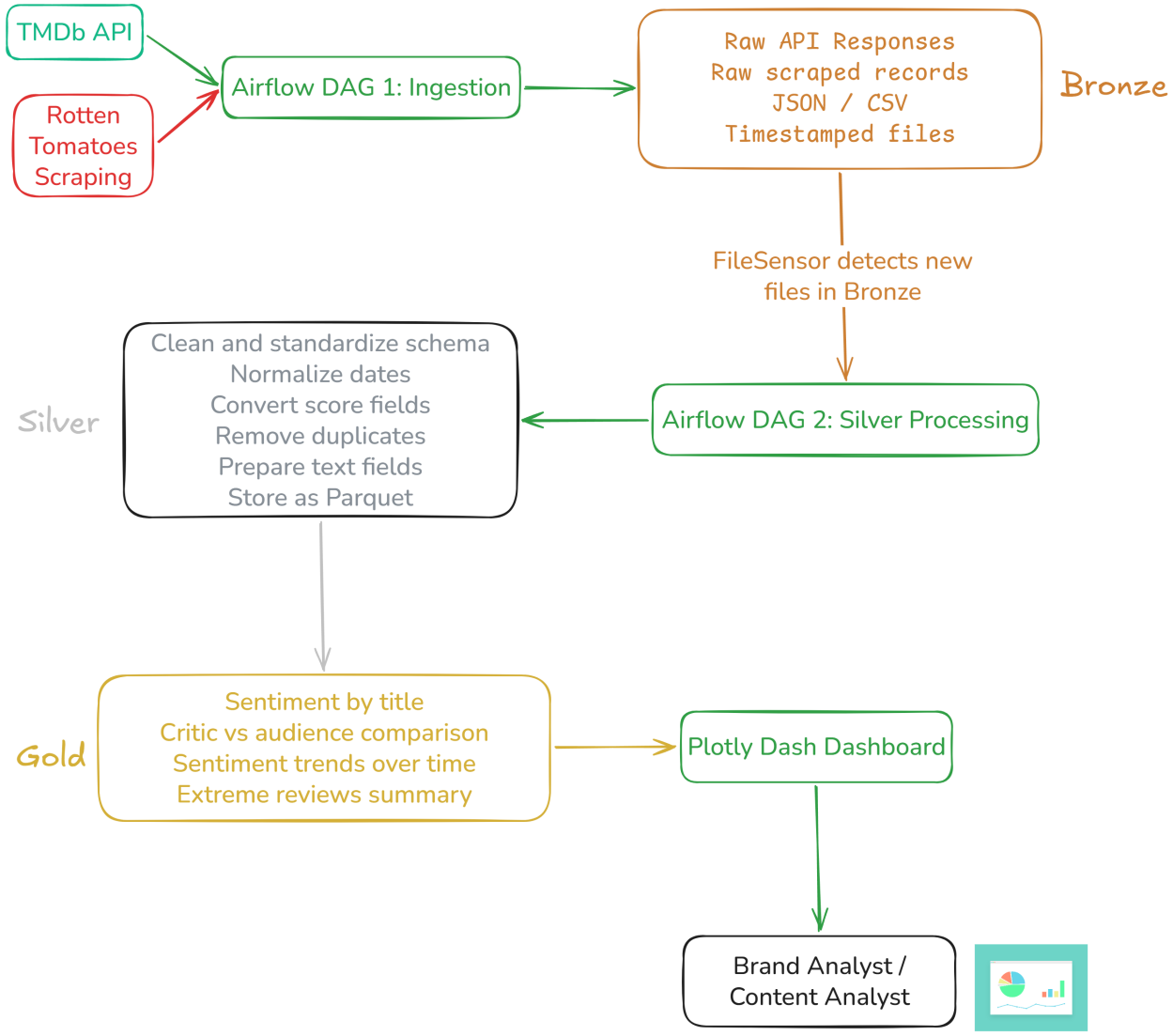

Figure 2: High-level architecture of the proposed data pipeline, showing the flow from source ingestion and raw storage to transformation, aggregation, and dashboard consumption.

References

[1] The Movie Database (TMDb), "TMDb API Documentation – Getting Started," 2026. [Online]. Available: https://developer.themoviedb.org/docs/getting-started. [Accessed: March 2026].

[2] The Movie Database (TMDb), "TMDb API Reference," 2026. [Online]. Available: https://developer.themoviedb.org/reference/getting-started. [Accessed: March 2026].

[3] Rotten Tomatoes, "Movies in Theaters – Browse Page," 2026. [Online]. Available: https://www.rottentomatoes.com/browse/movies_in_theaters/sort:popular. [Accessed: March 2026].

[4] Rotten Tomatoes, "robots.txt," 2026. [Online]. Available: https://www.rottentomatoes.com/robots.txt. [Accessed: March 2026].

[5] Apache Airflow, "Apache Airflow Documentation," 2026. [Online]. Available: https://airflow.apache.org/docs/. [Accessed: March 2026].

[6] Apache Airflow, "Core Concepts," 2026. [Online]. Available: https://airflow.apache.org/docs/apache-airflow/stable/core-concepts/index.html. [Accessed: March 2026].

[7] Plotly, "Dash Documentation & User Guide," 2026. [Online]. Available: https://dash.plotly.com/. [Accessed: March 2026].